

FA26: DATA-266 Sec 21 & 71 - Generative AI and LLM

Homework 5

Siva Surya Chandran
019130215

What I did

For this assignment I fine-tuned google/flan-t5-base (about 248M parameters) to summarize conversations, using LoRA through HuggingFace's PEFT library, on the DialogSum dataset (neil-code/dialogsum-test). I ran the model on the same 2 test dialogues before and after fine-tuning to see what changed, scored both on the whole test set with ROUGE, and at the end trained a second adapter with a smaller rank ($r=4$ vs $r=16$) to see how much the rank actually matters. Everything below - the code, all the outputs, and the write-up for each part - is in the notebook HW5.ipynb, which I ran top to bottom on a Google Colab T4 GPU.

The assignment says flan-t5-small "or similar". I went with flan-t5-base because small's summaries were too weak to show a meaningful difference before and after.

Getting the data ready

The dataset has 1,999 train, 499 validation and 499 test rows. I turned every row into an input/target pair: the input is the dialogue with "Summarize the following conversation." in front (FLAN-T5 was instruction-tuned, so it works better when you tell it the task), and the target is the human-written summary. Inputs are cut off at 512 tokens and summaries at 128. The median input is 213 tokens and only 45 of the 1,999 training dialogues are long enough to get cut.

One thing I want to flag: the test split has 499 rows but only 167 different dialogues. Every dialogue shows up 3 times (test_0_1, test_0_2, test_0_3), once per human summary. The first time I picked my 2 demo dialogues I accidentally picked the same conversation twice. So I grouped the test rows by dialogue, and every test dialogue is used once and scored against its 3 reference summaries.

LoRA setup

LoRA keeps all the original weights frozen and learns a small low-rank update for each chosen weight matrix, $\Delta W = (\alpha/r) \cdot BA$. Only A and B get trained, which is a tiny fraction of the model. I put the adapters on the q and v attention projections in every attention block (encoder and decoder).

Setting	Value
Base model	google/flan-t5-base
Rank r	16
lora_alpha	32 ($\alpha/r = 2$)
lora_dropout	0.05
Target modules	q, v

Setting	Value
Total parameters	249,347,328
Trainable parameters	1,769,472 (0.71%)
Saved adapter size	7.1 MB

Training

I wrote a plain PyTorch training loop on the full training split: AdamW with learning rate $1e-3$ (LoRA usually wants a higher learning rate than full fine-tuning since it updates so few weights), 5% warmup then linear decay, gradient clipping at 1.0, and an effective batch size of 8 (micro-batches of 4 with 2 gradient-accumulation steps). 3 epochs is 750 optimizer steps. I trained in fp32 because T5 is known to blow up to NaN in fp16 and the T4 doesn't support bf16. It took about 11 minutes.

I first tried running it on my Mac (16 GB), but training used up all the memory and slowed to a crawl, which is why I moved to Colab.

	Before	Epoch 1	Epoch 2	Epoch 3
Validation loss	1.83	1.12	1.10	1.08

Training loss drops fast in the first ~75 steps (1.73 to 1.30) and after that slowly goes down with a lot of step-to-step noise, ending at 1.14. Validation loss was still going down at epoch 3, so it wasn't overfitting yet.

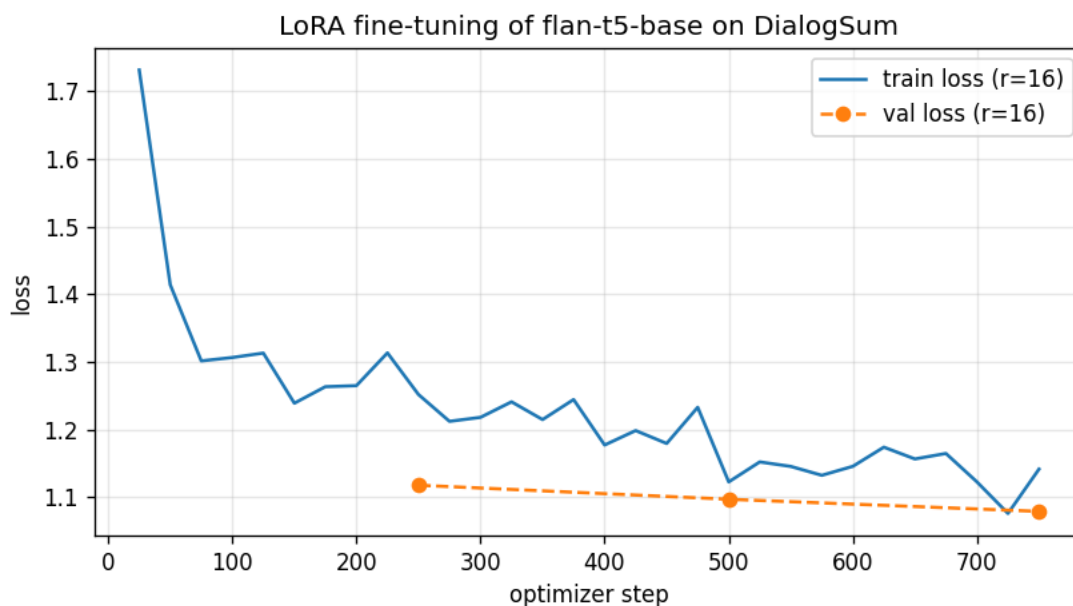


Figure 1: training and validation loss for LoRA $r=16$

Before vs after fine-tuning

Same 2 test dialogues (test_0 and test_1), same greedy decoding, before and after. The model never saw these during training.

test_0 - #Person1# dictates a memo to Ms. Dawson banning Instant Messaging at work:

Reference: Ms. Dawson helps #Person1# to write a memo to inform every employee that they have to change the communication method and should not use Instant Messaging anymore.

Before: The memo is to be distributed to all employees by this afternoon.

After (r=16): #Person1# asks Ms. Dawson to take a dictation for #Person1#. Ms. Dawson tells #Person1# the rules of office communications and the new policy.

test_1 - #Person2# is late because of traffic and gets talked into taking public transport:

Reference: #Person2# arrives late because of traffic jam. #Person1# persuades #Person2# to use public transportations to keep healthy and to protect the environment.

Before: The traffic jam at the Carrefour intersection has caused a lot of congestion.

After (r=16): #Person2# got stuck in traffic again. #Person1# suggests #Person2# start taking public transport system to work. #Person2# thinks it's better for the environment and #Person2# will quit driving to work.

Before fine-tuning, the model basically picks one detail from the conversation (the memo deadline, the traffic jam) and writes a single short sentence about it, missing the actual point. After fine-tuning, the summaries read like the DialogSum ones: they refer to people as #Person1#/#Person2#, go through the whole conversation, and for test_1 it gets the actual outcome.

It's still not perfect. In test_0 it mixes up who's doing what: it says Ms. Dawson tells #Person1# the rules, when it's really #Person1# dictating the memo to her. And it only says "the new policy" without saying what the policy is.

Two examples don't say much on their own, so I also scored both models with ROUGE on all 167 test dialogues (each prediction compared against the best of its 3 references):

Model	ROUGE-1	ROUGE-2	ROUGE-L	Avg words
flan-t5-base, no fine-tuning	28.37	11.09	24.99	15.5
LoRA r=16	50.38	24.52	42.34	21.5
Human references	-	-	-	19.3

ROUGE-1 went from 28.4 to 50.4, and the summaries got longer (15.5 to 21.5 words), closer to the humans' 19.3.

Experiment: LoRA rank r=4 vs r=16

For my experiment I trained a second adapter with r=4. Everything else is the same as the r=16 run: same data, seed, learning rate, epochs, batch size and target modules. The one thing I was careful about is alpha: I kept alpha/r fixed at 2, so r=4 gets alpha=8. If I left alpha at 32 for both, the r=4 updates would get scaled 4x more than the r=16 ones, and I'd be changing the effective learning rate along with the rank.

	r=4	r=16
lora_alpha	8	32
Trainable params	442,368 (0.18%)	1,769,472 (0.71%)
Adapter file size	1.8 MB	7.1 MB
Final train loss	1.2019	1.1418
Final val loss	1.0953	1.0794
ROUGE-1 / ROUGE-2 / ROUGE-L	49.50 / 23.41 / 41.32	50.38 / 24.52 / 42.34
Avg summary length	22.5 words	21.5 words
Training time (T4)	639 s	640 s



Figure 2: loss curves for r=4 vs r=16

r=16 was a little better on every number, but only a little. r=4 trains 4x fewer parameters and ends up only about 1 ROUGE point behind - it already gets almost the whole jump from the base model (ROUGE-1 28.4 to 49.5). Its loss curves follow the r=16 ones almost exactly, just slightly higher the whole way. Training time was basically the same, because most of the compute goes into running the frozen base model, so the adapter's size barely matters for speed.

When I read the actual summaries, I couldn't say r=16 is clearly better. The two give different summaries on 144 of the 167 test dialogues, but a lot of that is wording, and they often start with the exact same sentence. On test_1, r=4 throws in more details (the pollution, biking to work) but misses how it ends, while r=16 gets the actual decision. On test_0 it goes the other way: r=4 actually mentions Instant Messaging, which is the whole point of the memo, and r=16 stays vague.

test_0, r=4: Ms. Dawson tells #Person1# the new policy of using Instant Messaging in the office.

test_1, r=4: #Person2# got stuck in traffic again. #Person1# suggests #Person2# start taking public transport system to work. #Person2# feels bad about the pollution problem in this city. #Person1# recommends biking to work.

Both ranks make the same kind of mistake, and that's mixing up who said what:

- test_2: the reference says "#Person1# tells Kate that Masha and Hero get divorced", and both models say "Kate tells #Person2# ...".
- test_3: #Person1# asks Brian to dance, and both models say Brian invites #Person1# to have a dance with him.

Going up in rank didn't fix this, so my guess is it's more about the base model's size than the adapter's capacity.

One caveat: this is one seed and 167 test dialogues, so I wouldn't read too much into a 1-point gap. It does line up with r=16's lower train and val loss at every checkpoint, though. With val loss still going down at epoch 3, I think training longer would help more than raising the rank.

Quick summary

Model	Trainable params	ROUGE-1	ROUGE-L	Summary quality
No fine-tuning	-	28.37	24.99	One detail, misses the point
LoRA r=4	442,368	49.50	41.32	DialogSum style, mixes up speakers
LoRA r=16	1,769,472	50.38	42.34	DialogSum style, mixes up speakers